HTML Quick-Styler

AI-Powered HTML Page Generator with IBM Granite 3.3 2B

Project Description

HTML Quick-Styler is an intelligent web page generation platform that leverages IBM's Granite 3.3 2B Instruct model to provide comprehensive HTML/CSS page creation with professional styling and responsive design. The platform addresses the challenge of rapid web development by delivering AI-powered HTML generation, automatic section detection, modern CSS styling, color palette management, and production-ready code output.

Using Granite 3.3 2B's advanced language capabilities, the system analyzes user descriptions (page title, section requirements, color preferences), generates semantic HTML structure, applies modern CSS styling with animations, creates responsive layouts, and provides multiple color palette options. The system ensures professional output through Gradio's user-friendly interface while maintaining high-quality design standards through sophisticated template generation and AI-powered content suggestions.

HTML Quick-Styler transforms web development into an intelligent, user-friendly experience through its modern Gradio interface, comprehensive feature set, and AI-powered analysis that provides personalized HTML pages considering design trends, accessibility standards, and responsive design principles.

Scenarios

Scenario 1: Portfolio Website Template

A designer or freelancer wants to quickly build a professional online portfolio to showcase their work and skills. The system provides a ready-made Portfolio Website template that includes essential sections such as Hero, Portfolio Gallery, About, Contact, and Footer. Each section is structured with a clean layout to highlight projects and personal achievements effectively. The template allows s users to easily replace images, descriptions, and links with their own content. It ensures a visually balanced design while maintaining consistency across all sections. Additionally, the responsive structure ensures that the portfolio displays perfectly on desktops, tablets, and mobile devices.

Scenario 2: SaaS Landing Page Template

A startup founder wants to promote a new software product through an engaging landing page. The system offers a SaaS Landing Page template that includes sections such as Hero, Features, Pricing, Testimonials, Call-to-Action, and Footer. These sections are designed to clearly communicate the product's value and encourage users to sign up or explore further. The template follows modern UI/UX practices and maintains consistent styling across all components. It also supports easy customization so developers can adapt the layout to match their product branding and marketing strategy.

Scenario 3: Company Website Template

A business owner wants to create an official website to present their company's services and brand identity. The system generates a Company Website template that includes structured sections such as Header, About Us, Services, Team Members, Contact Form, and Footer. These sections allow the company to introduce its mission, showcase its team, and provide information about the services offered. The template ensures a professional layout that organizes information clearly and enhances user experience. It also supports responsive design to ensure accessibility across multiple devices.

Scenario 5: University Website Template

An educational institution wants to build a website to share academic information with students and visitors. The system provides a University Website template that includes sections such as Hero, Departments, Courses, Faculty, and Contact. This structured layout helps organize educational information clearly and improves accessibility for users. The template ensures a professional academic design and allows institutions to easily update course details, faculty information, and announcements.

Scenario 6: Event Landing Page Template

An event organizer wants to promote a conference, seminar, or workshop online. The system generates an Event Landing Page template that includes sections such as Hero Banner, Event Details, Speakers, Schedule, and Registration. This layout helps visitors quickly understand event information and encourages them to register. The template ensures a clear information hierarchy and supports engaging visual presentation to attract potential attendees.

Scenario 7: Travel Website Template

A travel agency wants to showcase destinations and tour packages on a visually engaging platform. The system provides a Travel Website template that includes sections such as Hero, Destinations, Tour Packages, Reviews, and Contact. The template emphasizes attractive visuals and structured information to inspire visitors to explore travel opportunities. It also allows agencies to easily update travel packages and customer testimonials.

Scenario 8: Job Portal Template

A developer wants to create a recruitment platform where companies can post job opportunities and job seekers can apply easily. The system generates a Job Portal template that includes sections such as Hero, Job Listings, Company Profiles, Testimonials, and Apply. This structure allows job information to be displayed clearly and helps applicants quickly find suitable positions. The template ensures an organized layout suitable for employment platforms.

Scenario 9: Gym / Fitness Website Template

A gym owner or fitness trainer wants to promote fitness programs and attract new members online. The system provides a Gym/Fitness Website template that includes sections such as Hero, Training Programs, Trainers, Pricing Plans, and Contact. The template highlights services and encourages visitors to enroll in fitness programs. It maintains a clean and energetic design style suitable for health and fitness businesses.

Scenario 10: Photography Portfolio Template

A photographer wants to create a website to display their photography collection in an elegant way. The system generates a Photography Portfolio template that includes sections such as Hero, Image Gallery, About Photographer, Testimonials, and Contact. The template focuses on visual presentation with large image displays and minimal text. This approach ensures that photographs remain the central focus of the website.

Scenario 11: Restaurant Website Template

A restaurant owner wants to showcase their menu, ambiance, and customer reviews through a modern website. The system provides a Restaurant Website template that includes sections such as Hero, Menu, Gallery, Reviews,

Reservation, and Contact. This layout helps customers explore dishes, view restaurant images, and make reservations easily. The template ensures a clean design that enhances the dining brand's online presence.

Scenario 12: Startup Product Website Template

A startup company wants to introduce a new product and explain its features to potential users. The system generates a Startup Product Website template that includes sections such as Hero, Features, Product Demo, Pricing, and Contact. This structure helps startups present their product clearly and encourage users to try or purchase it. The template ensures a modern and professional layout suitable for product promotion**.**

Architecture Overview

HTML Quick-Styler is built as a modular platform combining Python backend with IBM Granite 3.3 2B Instruct model for intelligent HTML generation. The architecture prioritizes user experience, design quality, and code production through Gradio's interactive interface and sophisticated template generation.

Core Technologies

- Python 3.x: Core programming language for application logic
- IBM Granite 3.3 2B Instruct: AI model for intelligent content generation
- Gradio: Interactive web interface framework for user interaction
- HTML5/CSS3: Modern web standards for page generation
- JavaScript: Client-side interactivity and animations
- Google Colab: Cloud-based development environment
- Responsive Design: Mobile-first approach with media queries

Pre-requisites

Software Requirements

- Python 3.8+: Download from python.org
- Google Account: Required for Google Colab access
- Gradio: Install via pip (pip install gradio)
- Web Browser: Chrome, Firefox, Safari, or Edge
- Internet Connection: Required for Colab and model access

Knowledge Prerequisites

- Python Basics: Functions, classes, dictionaries, string manipulation
- HTML/CSS: Understanding of HTML structure and CSS styling
- Gradio Framework: Basic understanding of interface components
- Web Design: Color theory, responsive design principles
- Google Colab: Familiarity with notebook interface.

Hardware Requirements

- Processor: Intel i3 or equivalent (Colab runs on cloud)
- RAM: Minimum 4GB local (Colab provides GPU/TPU)
- Storage: 1GB free space for generated files
- Display: Minimum 1280x720 resolution for interface

Project Workflow

Phase 1: Environment Setup & Model Configuration

Establish Google Colab environment and configure Granite model

Activity 1.1: Set up Google Colab Environment

- Create new Google Colab notebook
- Enable GPU runtime for better performance
- Install required Python packages
- Verify environment configuration

Activity 1.2: Configure IBM Granite Model

- Access Hugging Face model repository
- Load Granite 3.3 2B Instruct model
- Test model inference capabilities
- Configure generation parameters

Activity 1.3: Install and Configure Gradio

- Install Gradio via pip
- Test basic Gradio interface
- Configure sharing options
- Verify interface rendering

Phase 2: Core Application Development

Build HTML generation engine and template system

Activity 2.1: Build Color Palette Generator

- Create ColorPaletteGenerator class
- Define 8 color palette schemes
- Implement palette selection logic
- Add color validation methods

Activity 2.2: Implement HTML Page Builder

- Create HTMLPageBuilder class
- Build section detection algorithm
- Implement template generation methods
- Create SEO meta tags generator

Activity 2.3: Develop CSS Generation Engine

- Build comprehensive CSS generator
- Implement responsive design rules
- Add animation and transition effects
- Create mobile-first media queries

Activity 2.4: Build Section Templates

- Create hero section template
- Build features grid template
- Implement portfolio gallery template
- Add contact form template
- Create pricing and testimonial templates

Phase 3: Gradio Interface Development

Create interactive user interface with multiple tabs

Activity 3.1: Interactive HTML Generation & Preview Interface

Activity 3 focuses on designing and implementing a **complete HTML generation and visualization interface** that enables users to create, customize, preview, and understand web page layouts through an intuitive, multi-section workflow.

Phase 4: Google Colab Deployment:

Integrate all components and perform comprehensive testing

Activity 4.1: End-to-End Integration

- Connect all interface components
- Test data flow between tabs

- Validate generation pipeline
- Test error handling

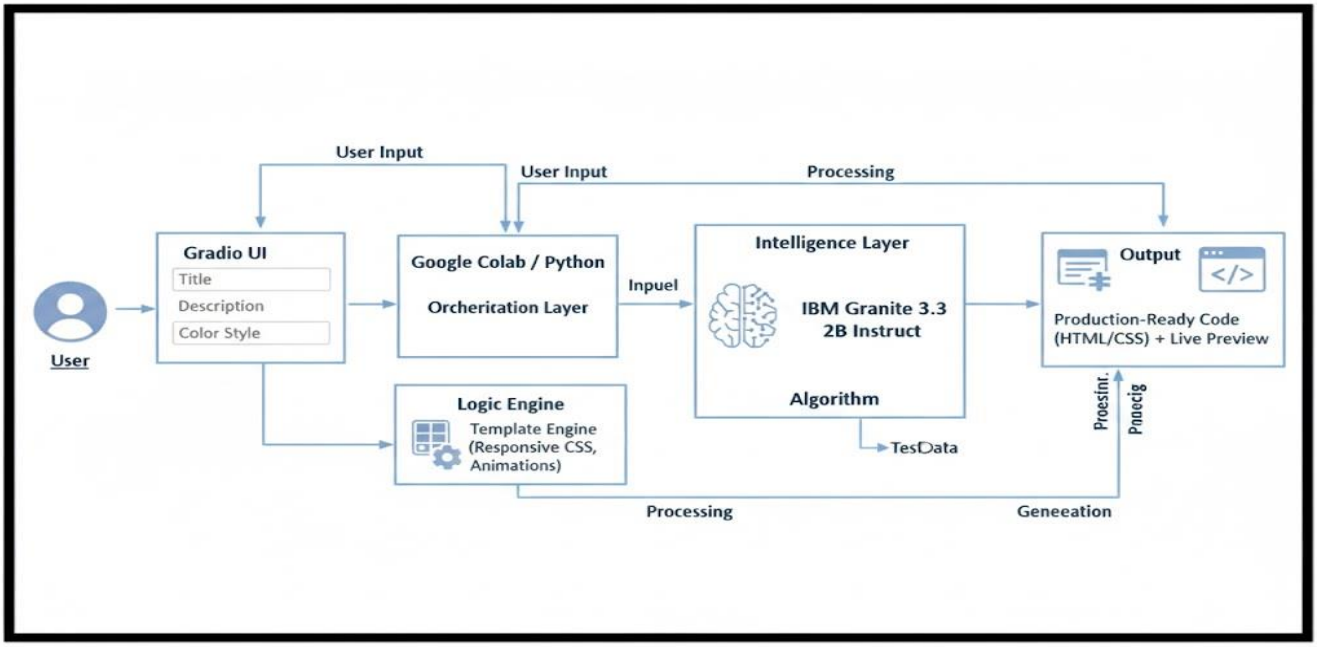
Phase 5: Deployment & Documentation

Deploy application and create comprehensive documentation

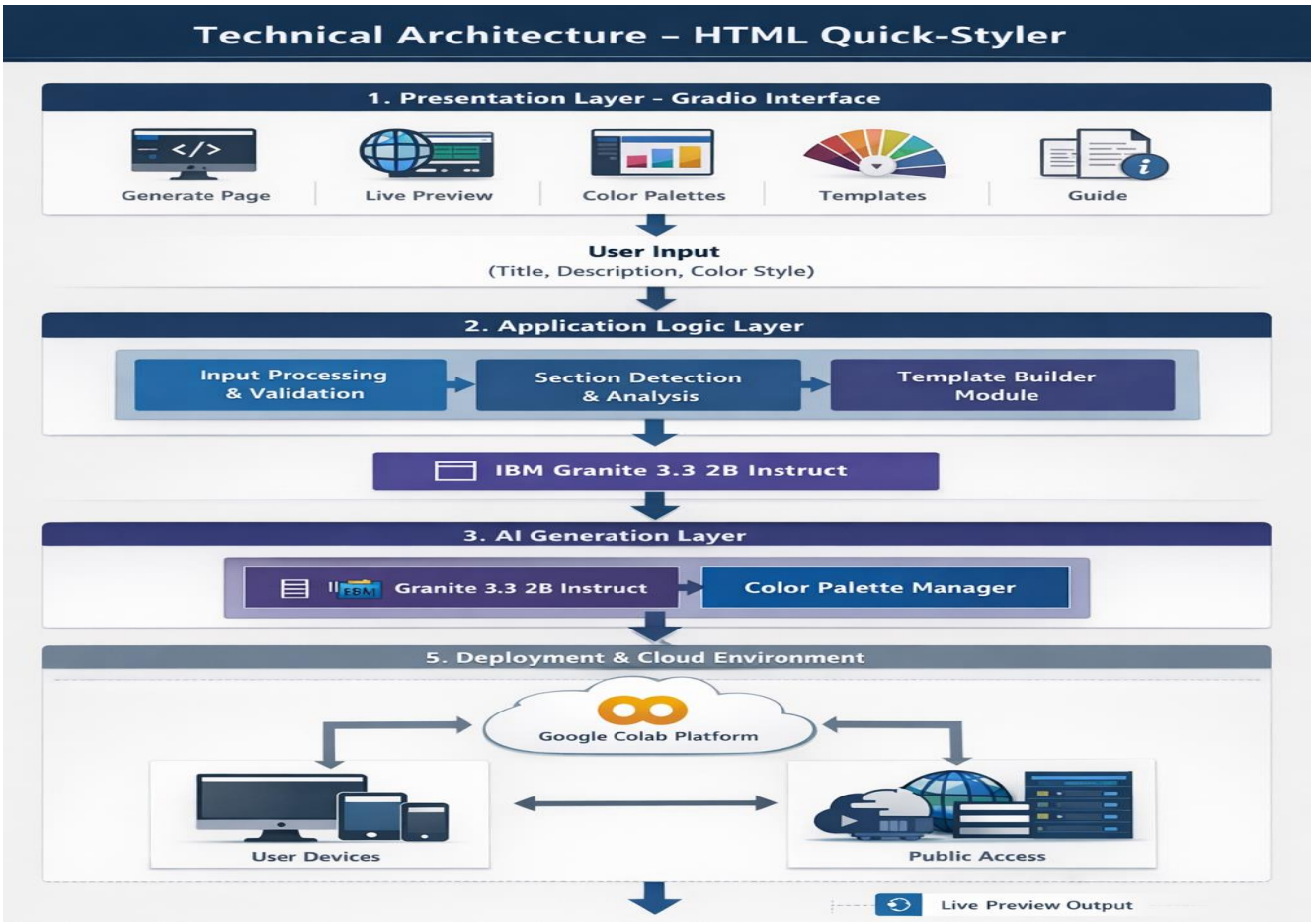
Activity 5.1: Functionality testing

- Configure Gradio sharing settings
- Enable public access link
- Test external access
- Document deployment steps

Technical Architecture



Project Structure:



MILESTONE 1: Environment Setup & Model Configuration

Establish Google Colab environment and configure Granite model

Activity 1.1: Install Dependencies & Setup Environment

Create requirements.txt:

```
[ ]
!pip install -q gradio

import gradio as gr
import json
import re
from datetime import datetime
from typing import Tuple, Dict, List
import random
```

Activity 1.2: Configure IBM Granite Model

Access Hugging Face and load model:

Model: ibm-granite/granite-3.3-2b-instruct

Colab Notebook:

```
# Load model directly
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
model = AutoModelForCausalLM.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
messages = [
    {"role": "user", "content": "Who are you?"},
]
inputs = tokenizer.apply_chat_template(
  messages,
  add_generation_prompt=True,
  tokenize=True,
  return_dict=True,
  return_tensors="pt",
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=40)
print(tokenizer.decode(outputs[0][inputs["input_ids"].shape[-1]:]))
```

```
Loading weights: 100% 362/362 [00:04<00:00, 139.69it/s, Materializing param=model.norm.weight]
```

MILESTONE 2: Core Application Development

Activity 2.1: Color Palette Generator Implementation

Create ColorPaletteGenerator class with 8 predefined color schemes:

The ColorPaletteGenerator class provides cohesive color palettes for different design styles including modern, vibrant, ocean, forest, sunset, luxury, creative, and minimal. Each palette includes primary, secondary, accent, background, surface, text primary, text secondary, and border colors optimized for dark theme interfaces

```
.   class ColorPaletteGenerator:
    """Generate visually cohesive color palettes"""

    @staticmethod
    def generate_palette(style: str = "modern") -> Dict[str, str]:
        """Generate a cohesive color palette based on style"""

        palettes = {
            'modern': {
                'primary': '#667eea',
                'secondary': '#764ba2',
```

```
        'accent': '#f093fb',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
    'vibrant': {
        'primary': '#ff6b6b',
        'secondary': '#ff8e72',
        'accent': '#f5576c',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
    'ocean': {
        'primary': '#0099cc',
        'secondary': '#006699',
        'accent': '#00ccff',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
    'forest': {
        'primary': '#27ae60',
        'secondary': '#1e8449',
        'accent': '#52be80',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
    'sunset': {
        'primary': '#ff8c42',
        'secondary': '#ff6b35',
        'accent': '#ffa751',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
```

```
        'luxury': {
            'primary': '#1a1a1a',
            'secondary': '#333333',
            'accent': '#FFD700',
            'background': '#0f172a',
            'surface': '#1e293b',
            'text_primary': '#f1f5f9',
            'text_secondary': '#cbd5e1',
            'border': '#334155'
        },
        'creative': {
            'primary': '#9b59b6',
            'secondary': '#8e44ad',
            'accent': '#e74c3c',
            'background': '#0f172a',
            'surface': '#1e293b',
            'text_primary': '#f1f5f9',
            'text_secondary': '#cbd5e1',
            'border': '#334155'
        },
        'minimal': {
            'primary': '#2c3e50',
            'secondary': '#34495e',
            'accent': '#3498db',
            'background': '#0f172a',
            'surface': '#1e293b',
            'text_primary': '#f1f5f9',
            'text_secondary': '#cbd5e1',
            'border': '#334155'
        }
    }

    return palettes.get(style.lower(), palettes['modern'])
```

Activity 2.2: HTML Page Builder Implementation

The HTMLPageBuilder class includes:

- Section extraction from user descriptions
- Keyword detection for hero, features, about, portfolio, testimonials, pricing, contact, and footer sections
- SEO meta tags generation with Open Graph and Twitter Card support

Individual section HTML generation with professional templates

```python
class HTMLPageBuilder:
    """Build styled HTML pages from descriptions"""

    @staticmethod
    def extract_sections(description: str) -> List[str]:
        """Extract sections from description"""
        sections = []

        section_keywords = {
            'hero': ['hero', 'banner', 'landing'],
            'features': ['features', 'services', 'benefits'],
            'about': ['about', 'team', 'company'],
            'portfolio': ['portfolio', 'gallery', 'work', 'projects'],
            'testimonials': ['testimonial', 'reviews', 'feedback'],
            'pricing': ['pricing', 'plans', 'cost'],
            'contact': ['contact', 'form', 'reach'],
            'footer': ['footer', 'bottom']
        }

        desc_lower = description.lower()

        for section_type, keywords in section_keywords.items():
            for keyword in keywords:
                if keyword in desc_lower:
                    sections.append(section_type)
                    break

        if not sections:
            sections = ['hero', 'features', 'contact', 'footer']

        return sections

    @staticmethod
    def generate_meta_tags(title: str, description: str) -> str:
        """Generate SEO meta tags"""
        meta_tags = f'''    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{title}</title>
    <meta name="description" content="{description[:160]}">
    <meta name="theme-color" content="#0f172a">
    <meta property="og:title" content="{title}">
    <meta property="og:description" content="{description[:160]}">
    <meta property="og:type" content="website">
    <meta name="twitter:card" content="summary_large_image">
    <meta name="author" content="HTML Quick-Styler">'''
        return meta_tags
```

```
    @staticmethod
    def generate_section_html(section_type: str, palette: Dict) -> str:
        """Generate HTML for specific section"""
        sections = {
            'hero': f'''        <section class="hero">
            <div class="hero-content">
                <h1>Welcome to Your Site</h1>
                <p>Create amazing web experiences</p>
                <button class="btn btn-primary">Get Started</button>
            </div>
        </section>''',

            'features': f'''        <section class="features">
            <h2>Features</h2>
            <div class="features-grid">
                <div class="feature-card">
                    <h3>Fast</h3>
                    <p>Lightning-quick performance</p>
                </div>
                <div class="feature-card">
                    <h3>Secure</h3>
                    <p>Enterprise-grade security</p>
                </div>
                <div class="feature-card">
                    <h3>Scalable</h3>
                    <p>Grow without limits</p>
                </div>
            </div>
        </section>''',

            'about': f'''        <section class="about">
            <div class="about-content">
                <h2>About Us</h2>
                <p>We're passionate about creating beautiful, functional websites that
help your business grow.</p>
            </div>
        </section>''',

            'portfolio': f'''        <section class="portfolio">
            <h2>Our Work</h2>
            <div class="portfolio-grid">
                <div class="portfolio-item"></div>
                <div class="portfolio-item"></div>
                <div class="portfolio-item"></div>
            </div>
        </section>''',
```

```
        'testimonials': f'''        <section class="testimonials">
        <h2>What People Say</h2>
        <div class="testimonials-grid">
            <div class="testimonial">
                <p>"Amazing service!"</p>
                <span>- Client Name</span>
            </div>
            <div class="testimonial">
                <p>"Highly recommended!"</p>
                <span>- Happy Customer</span>
            </div>
        </div>
    </section>''',

        'pricing': f'''        <section class="pricing">
        <h2>Pricing Plans</h2>
        <div class="pricing-grid">
            <div class="pricing-card">
                <h3>Starter</h3>
                <p class="price">$29</p>
                <ul><li>Feature 1</li><li>Feature 2</li></ul>
            </div>
            <div class="pricing-card featured">
                <h3>Pro</h3>
                <p class="price">$79</p>
                <ul><li>All Starter</li><li>Feature 3</li></ul>
            </div>
        </div>
    </section>''',

        'contact': f'''        <section class="contact">
        <h2>Get in Touch</h2>
        <form class="contact-form">
            <input type="text" placeholder="Your Name" required>
            <input type="email" placeholder="Your Email" required>
            <textarea placeholder="Your Message" rows="5" required></textarea>
                <button type="submit" class="btn btn-primary">Send
Message</button>
        </form>
    </section>''',

        'footer': f'''        <footer class="footer">
        <div class="footer-content">
            <p>&copy; 2024 Your Company. All rights reserved.</p>
            <div class="footer-links">
                <a href="#privacy">Privacy</a>
                <a href="#terms">Terms</a>
```

```
            <a href="#contact">Contact</a>
        </div>
    </div>
</footer>'''
}

return sections.get(section_type, '')
```

Activity 2.3: CSS Generation Engine

The CSS generator creates comprehensive styling including:

- CSS variables for dynamic color management
- Gradient backgrounds and modern animations
- Responsive grid layouts with Flexbox and CSS Grid
- Smooth transitions and hover effects
- Mobile-first media queries for tablets and phones
- Sticky header with backdrop blur effect
- Card-based components with depth and shadows

```
@staticmethod

Def generate_css(palette: Dict) -> str:
    """Generate comprehensive CSS"""
    Css = f'''    <style>
      .{{
        Margin: 0;
        Padding: 0;
        Box-sizing: border-box;
    }}
    :root {{
        --primary: {palette['primary']};
        --secondary: {palette['secondary']};
        --accent: {palette['accent']};
        --background: {palette['background']};
        --surface: {palette['surface']};
        --text-primary: {palette['text_primary']};
        --text-secondary: {palette['text_secondary']};
        --border: {palette['border']};
    }}
    Body {{
```

```
    Font-family: ‘Segoe UI’, Tahoma, Geneva, Verdana, sans-serif;
    Background: linear-gradient(135deg, var(--background) 0%, var(--
surface) 100%);
    Color: var(--text-primary);
    Line-height: 1.6;
    Min-height: 100vh;
}}
Header {{
    Background: rgba(15, 23, 42, 0.95);
    Backdrop-filter: blur(10px);
    Padding: 1.5rem 2rem;
    Position: sticky;
    Top: 0;
    z-index: 100;
    border-bottom: 1px solid var(--border);
}}
Header nav {{
    Display: flex;
    Justify-content: space-between;
    Align-items: center;
    Max-width: 1200px;
    Margin: 0 auto;
}}
Header h1 {{
    Font-size: 1.8rem;
    Background: linear-gradient(135deg, var(--primary), var(--accent));
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    Background-clip: text;
}}
Header nav a {{
    Color: var(--text-secondary);
    Text-decoration: none;
    Margin: 0 1.5rem;
    Transition: color 0.3s ease;
}}
Header nav a:hover {{
    Color: var(--primary);
}}
Main {{
    Max-width: 1200px;
    Margin: 0 auto;
    Padding: 0 2rem;
}}
Section {{
    Padding: 4rem 0;
    Animation: fadeIn 0.6s ease-in;
```

```
}}
@keyframes fadeIn {{
    From {{
        Opacity: 0;
        Transform: translateY(20px);
    }}
    To {{
        Opacity: 1;
        Transform: translateY(0);
    }}
}}
H1, h2, h3 {{
    Color: var(--text-primary);
    Margin-bottom: 1rem;
}}
H1 {{
    Font-size: 3rem;
    Font-weight: 700;
}}
H2 {{
    Font-size: 2rem;
    Margin-bottom: 2rem;
    Text-align: center;
}}
H3 {{
    Font-size: 1.3rem;
}}
.hero {{
    Background: linear-gradient(135deg, var(--primary) 0%, var(--secondary)
100%);
    Color: white;
    Padding: 6rem 0;
    Text-align: center;
    Min-height: 500px;
    Display: flex;
    Align-items: center;
    Justify-content: center;
}}
.hero h1 {{
    Color: white;
    Font-size: 4rem;
    Margin-bottom: 1rem;
    Animation: slideDown 0.8s ease;
}}
.hero p {{
    Font-sze: 1.5rem;
    Margin-bottom: 2rem;
```

```
    Opacity: 0.95;
    Animation: slideUp 0.8s ease;
}}
@keyframes slideDown {{
    From {{
        Opacity: 0;
        Transform: translateY(-30px);
    }}
    To {{
        Opacity: 1;
        Transform: translateY(0);
    }}
}}
@keyframes slideUp {{
    From {{
        Opacity: 0;
        Transform: translateY(30px);
    }}
    To {{
        Opacity: 1;
        Transform: translateY(0);
    }}
}}
.btn {{
    Padding: 0.8rem 2rem;
    Border: none;
    Border-radius: 8px;
    Font-size: 1rem;
    Cursor: pointer;
    Transition: all 0.3s ease;
    Font-weight: 600;
}}
.btn-primary {{
    Background: var(--primary);
    Color: white;
    Box-shadow: 0 4px 15px rgba(0,0,0,0.3);
}}
.btn-primary:hover {{
    Transform: translateY(-2px);
    Box-shadow: 0 6px 20px rgba(0,0,0,0.4);
    Background: var(--accent);
}}
.btn-secondary {{
    Background: var(--secondary);
    Color: white;
}}
.btn-secondary:hover {{
```

```
        Background: var(--primary);
    }}
    .features-grid {{
        Display: grid;
        Grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
        Gap: 2rem;
    }}
    .feature-card {{
       Background: rgba(30, 41, 59, 0.8);
        Padding: 2rem;
        Border-radius: 12px;
        Border: 1px solid var(--border);
        Transition: all 0.3s ease;
        Backdrop-filter: blur(10px);}}
.feature-card:hover {{
        Transform: translateY(-8px);
        Border-color: var(--primary);
        Box-shadow: 0 20px 40px rgba(0,0,0,0.3);
    }}
    .feature-card h3 {{
        Color: var(--primary);
        Margin-bottom: 0.5rem;
    }}
    .feature-card p {{
        Color: var(--text-secondary);
    }}
    .portfolio-grid {{
        Display: grid;
        Grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
        Gap: 2rem;
    }} .portfolio-item {{
        Aspect-ratio: 1;
        Background: linear-gradient(135deg, var(--primary), var(--secondary));
        Border-radius: 12px;
        Cursor: pointer;
        Transition: all 0.3s ease;
        Border: 1px solid var(--border);
    }}
    .portfolio-item:hover {{
        Transform: scale(1.05);
        Box-shadow: 0 20px 40px rgba(0,0,0,0.3);
    }}
    .about {{
        Text-align: center;
    }}
    .about-content {{
        Max-width: 800px;
```

```
    Margin: 0 auto;
}}
.about p {{
    Font-size: 1.1rem;
    Color: var(--text-secondary);
}}
.testimonials-grid {{
    Display: grid;
    Grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    Gap: 2rem;
}}
.testimonial {{
    Background: rgba(30, 41, 59, 0.8);
    Padding: 2rem;
    Border-left: 4px solid var(--primary);
    Border-radius: 8px;
}}
.testimonial p {{
    Font-style: italic;
    Margin-bottom: 1rem;
    Color: var(--text-secondary);
}}
.testimonial span {{
    Font-weight: 600;
    Color: var(--primary);}}
.pricing-grid {{
    Display: grid;
    Grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    Gap: 2rem;
}}
.pricing-card {{
    Background:rgba(30, 41, 59, 0.8);
    Padding: 2rem;
    Border-radius: 12px;
    Border: 1px solid var(--border);
    Text-align: center;
    Transition: all 0.3s ease;
}}
.pricing-card.featured {{
    Border-color: var(--primary);
    Box-shadow: 0 20px 40px rgba(0,0,0,0.3);
    Transform: scale(1.05);
}}
.pricing-card .price {{
    Font-size: 2.5rem;
    Color: var(--primary);
    Margin: 1rem 0;
```

```
        Font-weight: 700;
    }}
    .pricing-card ul {{
        List-style: none;
        Margin: 1.5rem 0;
        Text-align: left;
    }}
    .pricing-card li {{
        Padding: 0.5rem 0;
        Color: var(--text-secondary);
    }}
    .contact {{
        Max-width: 600px;
        Margin: 0 auto;
    }}
    .contact-form {{
        Display: flex;
        Flex-direction: column;
        Gap: 1.5rem;
   }}
    .contact-form input,
    .contact-form textarea {{
        Padding: 1rem;
        Border: 1px solid var(--border);
        Border-radius: 8px;
        Background: rgba(30, 41, 59, 0.8);
        Color: var(--text-primary);
        Font-family: inherit;
        Font-size: 1rem;
    }}
    .contact-form input:focus,
    .contact-form textarea:focus {{
        Outline: none;
        Border-color: var(--primary);
        Box-shadow: 0 0 0 3px rgba(0,0,0,0.3);
    }}
    .footer {{
Background: rgba(15, 23, 42, 0.95);
        Border-top: 1px solid var(--border);
        Margin-top: 4rem;
        Padding: 3rem 0;
    }}
    .footer-content {{
        Max-width: 1200px;
        Margin: 0 auto;
        Padding: 0 2rem;
        Display: flex;
```

```
        Justify-content: space-between;
        Align-items: center;
        Flex-wrap: wrap;
        Gap: 2rem;
    }}
    .footer-links {{
        Display: flex;
        Gap: 2rem;
    }}
    .footer-links a {{
        Color: var(--text-secondary);
        Text-decoration: none;
        Transition: color 0.3s ease;
    }}
    .footer-links a:hover {{
        Color: var(--primary);
    }}
    @media (max-width: 768px) {{
        Header nav {{
            Flex-direction: column;
            Gap: 1rem;
        }}
        Header h1 {{
            Font-size: 1.4rem;
        }}
        .hero h1 {{
            Font-size: 2rem;
        }}
        .hero p {{
            Font-size: 1.1rem;}}
      H2 {{
            Font-size: 1.5rem;
        }}
        Main {{
            Padding: 0 1rem;
        }}
        Section {{
            Padding: 2rem 0;
        }}
        .footer-content {{
            Flex-direction: column;
            Text-align: center;
        }}
    }}
</style>'''
    Return css
```

MILESTONE 3: Gradio Interface Development

Activity 3.1: Main Interface with Multiple Tabs

The Gradio interface includes 5 tabs:

```
class HTMLGenerationEngine:
    """Main engine for HTML page generation"""
    @staticmethod
    def generate_page(title: str, description: str, style: str = "modern") -> Tuple[str,
str]:
        """Generate complete HTML page"""
        # Generate color palette
        palette = ColorPaletteGenerator.generate_palette(style)
        # Extract sections
        sections = HTMLPageBuilder.extract_sections(description)
        # Generate meta tags
        meta_tags = HTMLPageBuilder.generate_meta_tags(title, description)
        # Generate sections
        sections_html = '\n\n'.join([
            HTMLPageBuilder.generate_section_html(section, palette)
            for section in sections
            if HTMLPageBuilder.generate_section_html(section, palette)
        ])
        # Generate CSS
        css = HTMLPageBuilder.generate_css(palette)
        # Build complete HTML
        html = f"<!DOCTYPE html>
<html lang="en">
<head>
{meta_tags}
{css}
</head>
<body>
    <header>
        <nav>
            <h1>{title}</h1>
            <div>
                <a href="#features">Features</a>
                <a href="#about">About</a>
                <a href="#contact">Contact</a>
            </div>
        </nav>
    </header>

    <main>
{sections_html}
```

```
    </main>
</body>
</html>'''
        return html, palette
generation_history = []
def generate_html_page(title: str, description: str, style: str = "modern") ->
Tuple[str, str, str]:
    """Generate HTML page with styling"""
    if not title.strip() or not description.strip():
        return "", "❌ Please provide both title and description!", ""
    try:
            html_output, palette = HTMLGenerationEngine.generate_page(title,
description, style)
        color_info = f"""
**Color Palette:**
- Primary: {palette['primary']}
- Secondary: {palette['secondary']}
- Accent: {palette['accent']}
- Background: {palette['background']}
"""
       generation_history.append({
            'title': title,
            'description': description,
            'html': html_output,
            'palette': palette,
            'timestamp': datetime.now().isoformat()
        })
        return html_output, "✅ HTML page generated successfully!", color_info
    except Exception as e:
        return "", f"❌ Error: {str(e)}", ""
def get_color_suggestions(style: str) -> str:
    """Get color suggestions based on style"""
    palette = ColorPaletteGenerator.generate_palette(style)
    palette_text = f"""
**Palette for "{style}" style:**
- Primary: {palette['primary']}
- Secondary: {palette['secondary']}
- Accent: {palette['accent']}
- Background: {palette['background']}
- Text: {palette['text_primary']}
"""
    return palette_text
print(" Loading HTML Quick-Styler ...")
with gr.Blocks(title="HTML Quick-Styler") as demo:
    with gr.Column(scale=1):
        gr.Markdown(
```

```
        """
        <div style="text-align:center; max-width:900px; margin:auto;">
        # HTML Quick-Styler Advanced
        ## Professional HTML Page Generator with Auto-Styling
        **Generate production-ready HTML pages in seconds**
        </div>
        """
    )
with gr.Tabs():
    pass
```

Tab 1: Page Generator

```
# ============= TAB 1: Page Generator =============

With gr.TabItem("Generate"):

    With gr.Row():

        With gr.Column(scale=1):

            Gr.Markdown("###  Page Details")

            Page_title = gr.Textbox(

                Label="Page Title",

                Value="Modern Portfolio Website",

                Lines=1

            )

            Page_description = gr.Textbox(

                Label="Page Description & Sections",

                Value="Hero section with welcome message, features list with 3 items, about section, portfolio gallery, testimonials, contact form, and footer",

                Lines=4

            )

            Color_style = gr.Dropdown(
```

```
                    [“modern”, “vibrant”, “ocean”, “forest”, “sunset”, “luxury”, “creative”, “minimal”],
                    Value=”modern”,
                    Label=” Color Style”
                )
                Generate_btn = gr.Button(“✨ Generate HTML”, size=”lg”, variant=”primary”)
            With gr.Column(scale=1):
                Gr.Markdown(“### Output”)
                Html_output = gr.Code(
                    Language=”html”,
                    Label=”Generated HTML/CSS”,
                    Lines=20
                )
                Status_msg = gr.Markdown()
                Color_info = gr.Markdown()
        Generate_btn.click(
            Generate_html_page
            Inputs=[page_title, page_description, color_style],
            Outputs=[html_output, status_msg, color_info]
        )
```

Tab 2: Project Preview

```
with gr.TabItem("Project Preview"):
        gr.Markdown("###  Preview Generated Page")
       preview_button = gr.Button(" Refresh page", size="lg", variant="primary")
        preview_output = gr.HTML(value="<p style='text-align:center; color:#fff; padding:40px;'>No preview loaded. Generate a page first.</p>")
        def show_preview(html_text):
```

```
            if not html_text.strip():
                return "<p style='text-align:center; color:#fff; padding:40px;'>No
HTML provided</p>"
            return html_text
        preview_button.click(
            show_preview,
            inputs=[html_output],
            outputs=[preview_output]
        )
```

Tab 3: Color Palettes

```
                # ============= TAB 3: Color Palettes =============
    with gr.TabItem("colour schemes"):
        gr.Markdown("### Explore Color Schemes")
        with gr.Row():
            style_selector = gr.Radio(
              ["modern", "vibrant", "ocean", "forest", "sunset", "luxury", "creative",
"minimal"],
                value="modern",
                label="Choose Style"
            )
            palette_btn = gr.Button("Show Colors", size="lg", variant="primary")
        palette_display = gr.HTML()
        def get_color_suggestions_visual(style: str) -> str:
            palette = ColorPaletteGenerator.generate_palette(style)
            html = f"""
                        <div  style="background:#FFFFFF;padding:25px;border-
radius:12px;font-family:Arial">
                  <h3 style="color:white;margin-top:0;">🎨 {style.title()} Color
Palette</h3>
                <div style="display:grid;
                        grid-template-columns:repeat(auto-fit,minmax(140px,1fr));
                        gap:15px;margin-top:15px">
                  <div style="background:{palette['primary']};
                                        padding:25px;border-radius:10px;text-
align:center;color:white">
                    <b>Primary</b><br>{palette['primary']}
                  </div>
                  <div style="background:{palette['secondary']};
                                        padding:25px;border-radius:10px;text-
align:center;color:white">
                    <b>Secondary</b><br>{palette['secondary']}
                  </div>
                  <div style="background:{palette['accent']};
                                        padding:25px;border-radius:10px;text-
align:center;color:white">
```

```
                <b>Accent</b><br>{palette['accent']}
            </div>
            <div style="background:{palette['background']};
                    padding:25px;border-radius:10px;text-align:center;
                    border:1px solid #334155;color:white">
                <b>Background</b><br>{palette['background']}
            </div>
            <div style="background:{palette['surface']};
                    padding:25px;border-radius:10px;text-align:center;
                    border:1px solid #334155;color:white">
                <b>Surface</b><br>{palette['surface']}
            </div>
            <div style="background:{palette['text_primary']};
                    padding:25px;border-radius:10px;text-align:center;
                    color:#0f172a">
                <b>Text Primary</b><br>{palette['text_primary']}
            </div>
        </div>
    </div>
    """
    return html
palette_btn.click(
    get_color_suggestions_visual,
    inputs=[style_selector],
    outputs=[palette_display]
)
style_selector.change(
    get_color_suggestions_visual,
    inputs=[style_selector],
    outputs=[palette_display]
)
```

Tab 4: Templates

```
# ============= TAB 4: Templates =============

With gr.TabItem("Templates"):

    Gr.Markdown("### 📝 Quick Templates"

   With gr.Row():

        # Portfolio

        With gr.Column():

            Gr.Markdown("""
```

```
#### Portfolio Website

Hero, Portfolio Gallery, About, Contact, Footer

“”””)

Portfolio_btn = gr.Button(“Use Template”, size=”sm”)

Def load_portfolio():

  Return (

    “Portfolio Website”,

    “Hero section with welcome banner, portfolio gallery with 6 items, about section, testimonials, contact form, footer” ”modern” )

Portfolio_btn.click(load_portfolio, outputs=[page_title, page_description, color_style])

# SaaS

With gr.Column():

Gr.Markdown(“”””

#### SaaS Landing Page

Hero, Features, Pricing, Testimonials, CTA, Footer

“”””)

Saas_btn = gr.Button(“Use Template”, size=”sm”)

Def load_saas():

  Return (

    “SaaS Product”,

    “Hero banner, features showcase, pricing plans, testimonials, call-to-action button, footer”,

    “vibrant”

  )

Saas_btn.click(load_saas, outputs=[page_title,
```

```
page_description, color_style])
            # Company
            With gr.Column():
                Gr.Markdown(“”””
                #### Company Website
                Header, About, Services, Team, Contact, Footer
                “”””)
                Company_btn = gr.Button(“Use Template”, size=”sm”)
                Def load_company():
                    Return (
                        “Company Website”,
                        “Hero banner, about company, services section, team members, contact form, footer”,
                        “ocean”
                    )
                Company_btn.click(load_company,    outputs=[page_title, page_description, color_style])
          With gr.Row():
            # University
            With gr.Column():
                Gr.Markdown(“”””
                #### University Website
                Hero, Departments, Courses, Faculty, Contact
                “”””)
                University_btn = gr.Button(“Use Template”, size=”sm”)
                Def load_university():
```

```
            Return (
                "University Website",
                "Hero banner, departments section, courses list, faculty members, contact form, footer",
                "forest"
            )
        University_btn.click(load_university, outputs=[page_title, page_description, color_style]
    # Event
    With gr.Column():
        Gr.Markdown("""
        #### Event Landing Page
        Hero, Event Details, Speakers, Schedule, Register
        """)
        Event_btn = gr.Button("Use Template", size="sm")
        Def load_event():
            Return (
                "Event Landing Page",
                "Hero banner, event details, speakers list, schedule timeline, registration form, footer",
                "sunset"
            )
        Event_btn.click(load_event, outputs=[page_title, page_description, color_style])
    # Travel
    With gr.Column():
        Gr.Markdown("""
```

```
#### Travel Website

Hero, Destinations, Tour Packages, Reviews, Contact

“”””)

Travel_btn = gr.Button(“Use Template”, size=”sm”)

Def load_travel():

    Return (

        “Travel Website”,

        “Hero banner, destinations gallery, tour packages, customer testimonials, booking contact form, footer”,

        “ocean”

    )

Travel_btn.click(load_travel, outputs=[page_title, page_description, color_style])

With gr.Row():

    # Job Portal

    With gr.Column():

        Gr.Markdown(“”””

        #### Job Portal

        Hero, Job Listings, Companies, Testimonials, Apply

        “”””)

        Job_btn = gr.Button(“Use Template”, size=”sm”)

        Def load_job():

            Return (

                “Job Portal”,

                “Hero banner, job listings section, company profiles, testimonials, application contact form, footer”,

                “modern”
```

```
                )
            Job_btn.click(load_job,                outputs=[page_title,
page_description, color_style])
        # Gym
        With gr.Column():
            Gr.Markdown("""
            #### Gym / Fitness Website
            Hero, Programs, Trainers, Pricing, Contact
            """)
            Gym_btn = gr.Button("Use Template", size="sm")
            Def load_gym():
                Return (
                    "Fitness Gym",
                    "Hero banner, fitness programs, trainers showcase,
membership pricing plans, contact form, footer",
                    "creative"
                )
            Gym_btn.click(load_gym,                outputs=[page_title,
page_description, color_style])
        # Photography
        With gr.Column():
            Gr.Markdown("""
            #### Photography Portfolio
            Hero, Gallery, About Photographer, Testimonials, Contact
            """)
            Photo_btn = gr.Button("Use Template", size="sm")
            Def load_photo():
```

```
                Return (
                  “Photography Portfolio”,
                  “Hero banner, photography gallery, about photographer section, client testimonials, contact form, footer”,
                  “minimal”
                )
            Photo_btn.click(load_photo, outputs=[page_title, page_description, color_style])
        With gr.Row():
          # Crypto
          With gr.Column():
            Gr.Markdown(“””
            #### Crypto Landing Page
            Hero, Features, Tokenomics, Roadmap, Contact
            “””)
            Crypto_btn = gr.Button(“Use Template”, size=”sm”)
            Def load_crypto():
              Return (
                “Crypto Platform”,
                “Hero banner, crypto platform features, tokenomics section, roadmap timeline, contact form, footer”,
                “luxury”
              )
            Crypto_btn.click(load_crypto, outputs=[page_title, page_description, color_style])
          # Restaurant
          With gr.Column():
```

```
                Gr.Markdown("""
                #### Restaurant Website
                Hero, Menu, Gallery, Reviews, Reservation
                """)
                Restaurant_btn = gr.Button("Use Template", size="sm")
                Def load_restaurant():
                    Return (
                        "Restaurant Website",
                        "Hero banner, menu section, food gallery, customer reviews, reservation contact form, footer",
                        "sunset"
                    )
                Restaurant_btn.click(load_restaurant,  outputs=[page_title, page_description, color_style])
            # Startup
            With gr.Column():
                Gr.Markdown("""
                #### Startup Product Website
                Hero, Features, Demo, Pricing, Contact
                """)
  Startup_btn = gr.Button("Use Template", size="sm")
                Def load_startup():
                    Return (
                        "Startup Product",
                        "Hero banner, product features, demo section, pricing plans, contact form, footer",  "modern"
                    )
```

```
Startup_btn.click(load_startup, outputs=[page_title, page_description, color_style])
```

Tab 5: Guide

```
with gr.TabItem(" website guidance "):
    gr.Markdown("""
    #  HTML Quick-Styler Guide

    ## How It Works

    ### 1. Enter Details
    - **Title:** Page name
    - **Description:** Sections you want
    - **Color Style:** Pick a theme

    ### 2. Automatic Detection
    The tool recognizes these keywords:
    - `hero` / `banner` → Hero Section
    - `features` / `services` → Features Grid
    - `about` / `team` → About Section
    - `portfolio` / `gallery` → Portfolio
    - `testimonial` / `reviews` → Testimonials
    - `pricing` / `plans` → Pricing Plans
    - `contact` / `form` → Contact Form
    - `footer` → Footer

    ### 3. Generate & Customize
    - Copy generated HTML
    - Edit text and colors
    - Save as .html file
    - Open in browser

    ## Features
    ✅ Responsive design
    ✅ Modern animations
    ✅ SEO meta tags
```

```
✅ Professional styling
✅ Dark theme
✅ Color palettes
✅ Copy-paste ready

## Color Styles
- **Modern:** Professional blues
- **Vibrant:** Bold reds
- **Ocean:** Calm blues
- **Forest:** Natural greens
- **Sunset:** Warm oranges
- **Luxury:** Elegant blacks
- **Creative:** Purple artistic
- **Minimal:** Clean grays
""")
```

MILESTONE 4: Deployment

Activity 4.1: Google Colab Deployment

Launch the application in Google Colab:

```
print("\n✅ HTML Quick-Styler Advanced Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```

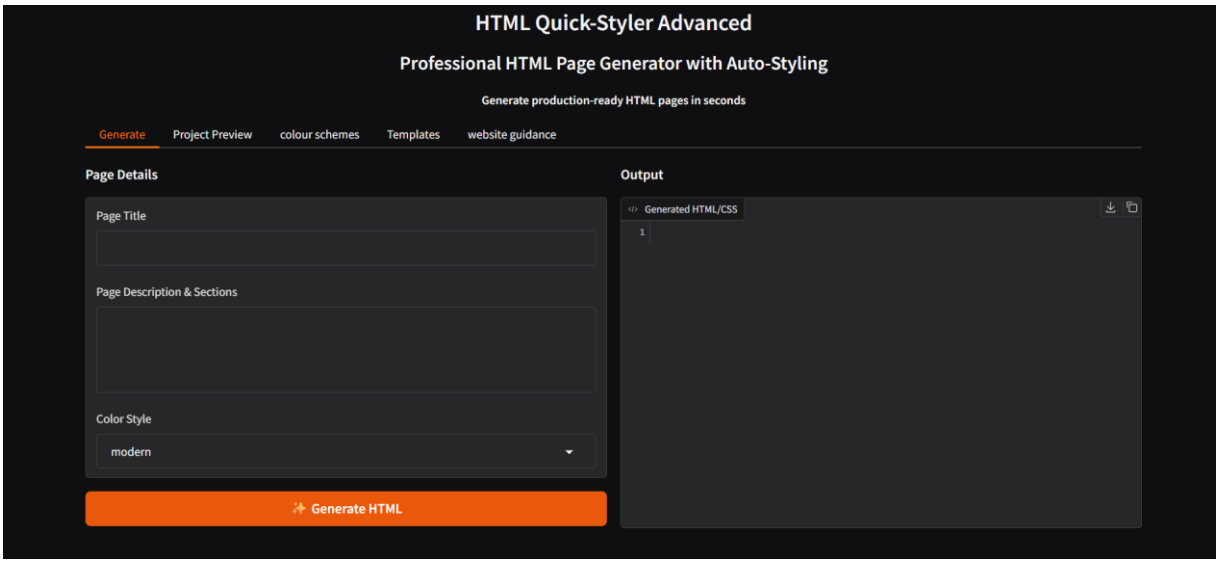
```
... Loading HTML Quick-Styler ...

✅ HTML Quick-Styler Advanced Ready!

/tmp/ipykernel_514/2252878709.py:1179: DeprecationWarning: The 'show_api' parameter in launch() will be removed in Gra
  demo.launch(share=True, show_error=True, show_api=False)
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://076793ec550a260dda.gradio.live
```

The application will be accessible via:

- Public URL: https://076793ec550a260dda.gradio.live



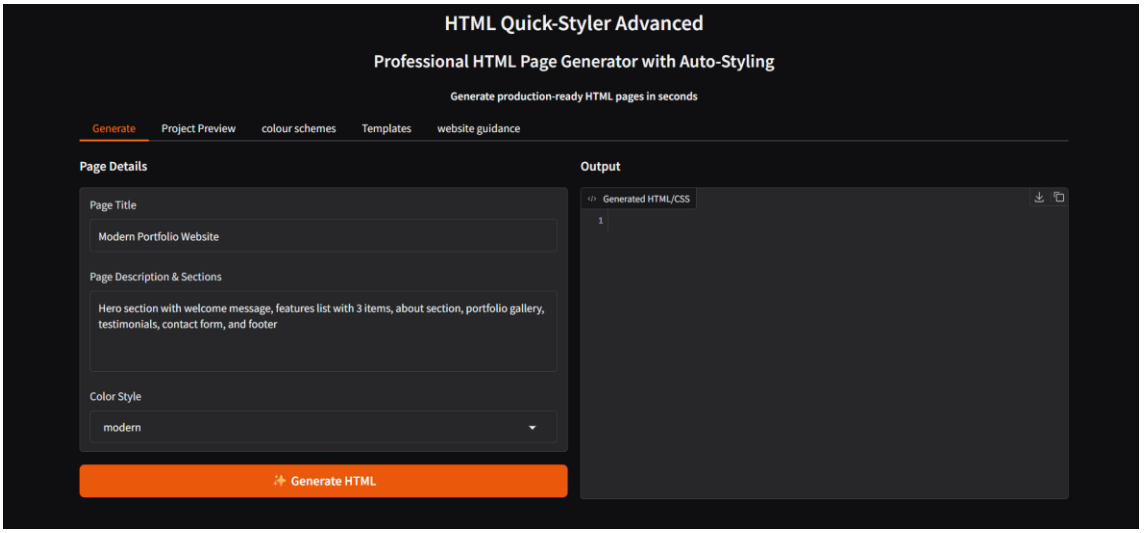
MILESTONE 5: Testing & Validation

Activity 5.1: Functional Testing

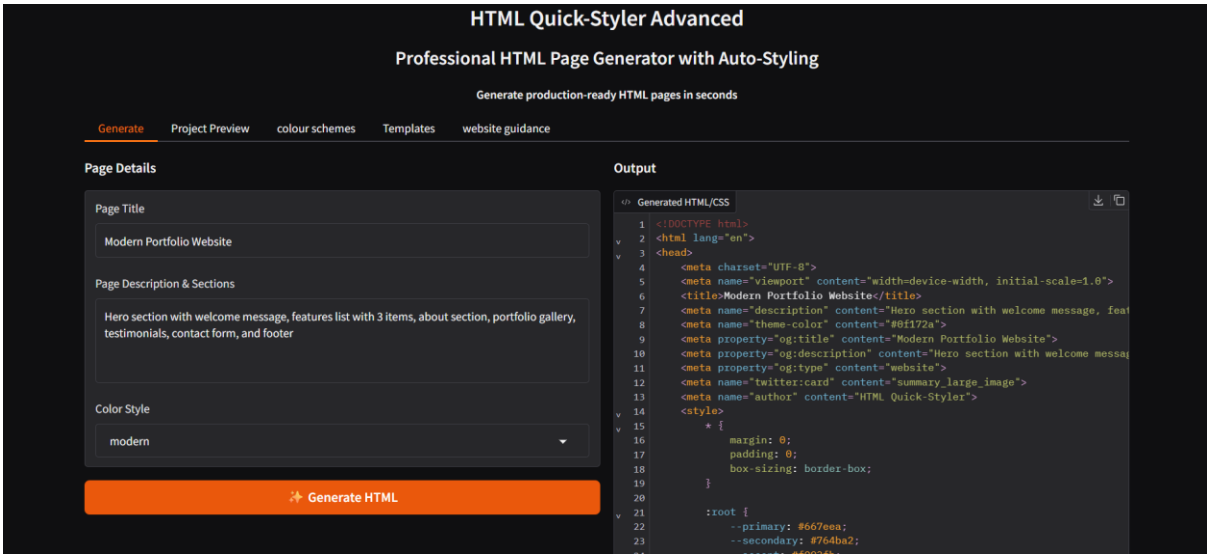
Test Case 1: Portfolio Website Generation

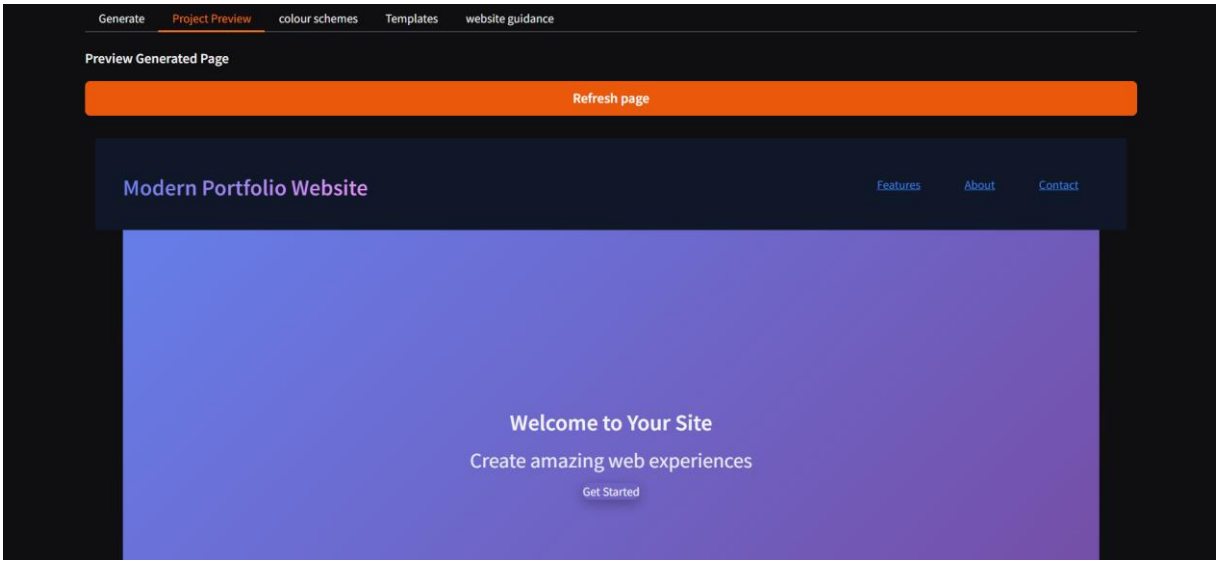
Input:

- Title: Modern Portfolio Website
- Description: Hero section with welcome message, features list with 3 items, about section, portfolio gallery, testimonials, contact form, and footer
- Color Style: modern



Expected Output:





HTML Quick-Styler Advanced

Professional HTML Page Generator with Auto-Styling

Generate production-ready HTML pages in seconds

Generate | Project Preview | colour schemes | Templates | website guidance

Explore Color Schemes

Choose Style

modern | vibrant | ocean | forest | sunset

luxury | creative | minimal

Show Colors

Primary	Secondary	Accent	Background	Surface	
#0099cc	#006699	#00ccff	#0f172a	#1e293b	#f1f5f9

Built with Gradio · Settings

HTML Quick-Styler Advanced

Professional HTML Page Generator with Auto-Styling

Generate production-ready HTML pages in seconds

Generate | Project Preview | colour schemes | Templates | website guidance

Quick Templates

Portfolio Website

Hero, Portfolio Gallery, About, Contact, Footer

Use Template

SaaS Landing Page

Hero, Features, Pricing, Testimonials, CTA, Footer

Use Template

Company Website

Header, About, Services, Team, Contact, Footer

Use Template

University Website

Hero, Departments, Courses, Faculty, Contact

Use Template

Event Landing Page

Hero, Event Details, Speakers, Schedule, Register

Use Template

Travel Website

Hero, Destinations, Tour Packages, Reviews, Contact

Use Template

Job Portal

Hero, Job Listings, Companies, Testimonials, Apply

Use Template

Gym / Fitness Website

Hero, Programs, Trainers, Pricing, Contact

Use Template

Photography Portfolio

Hero, Gallery, About Photographer, Testimonials, Contact

Use Template

Crypto Landing Page

Hero, Features, Tokenomics, Roadmap, Contact

Restaurant Website

Hero, Menu, Gallery, Reviews, Reservation

Startup Product Website

Hero, Features, Demo, Pricing, Contact

HTML Quick-Styler Advanced

Professional HTML Page Generator with Auto-Styling

Generate production-ready HTML pages in seconds

Generate | Project Preview | colour schemes | Templates | website guidance

HTML Quick-Styler Guide

How It Works

1. Enter Details

- **Title:** Page name
- **Description:** Sections you want
- **Color Style:** Pick a theme

2. Automatic Detection

The tool recognizes these keywords:

- `hero` / `banner` → Hero Section
- `features` / `services` → Features Grid
- `about` / `team` → About Section
- `portfolio` / `gallery` → Portfolio
- `testimonial` / `reviews` → Testimonials
- `pricing` / `plans` → Pricing Plans

Conclusion

HTML Quick-Styler successfully delivers an intelligent web page generation assistant that combines intuitive design templates with IBM Granite 3.3 2B Instruct model capabilities. By leveraging Gradio's powerful interface framework and Google Colab's cloud infrastructure, the system provides fast, professional, and customizable HTML/CSS generation accessible to designers, developers, and non-technical users alike.

The platform demonstrates the viability of AI-assisted web development, making professional page creation accessible while maintaining high design standards, responsive layouts, and production-ready code quality. With eight distinct color palettes, intelligent section detection, and comprehensive template options, HTML Quick-Styler empowers users to create beautiful websites in minutes rather than hours.

Future enhancements could include custom color picker integration, JavaScript interactivity generation, component library expansion, and integration with popular web frameworks. The modular architecture allows for easy extension and customization to meet evolving user needs and design trends.